Repaso Examen — Herencia, Clases Abstractas e Interfaces

HERENCIA (`extends`)

Una subclase **hereda automáticamente** atributos y métodos de la superclase y puede añadir los suyos.

```
class Electrodomestico {
    protected String marca;
    protected double potencia;
    public Electrodomestico(String marca, double potencia) {
        this.marca = marca;
        this.potencia = potencia;
    }
    public double getConsumo(int horas) { return potencia * horas; }
}

class Lavadora extends Electrodomestico {
    private boolean aguaCaliente;
    public Lavadora(String marca, double potencia, boolean aguaCaliente) {
        super(marca, potencia);         // OBLIGATORIO: primera línea, llama al padre
        this.aguaCaliente = aguaCaliente;
    }
    @Override
    public double getConsumo(int horas) {
        double pot = aguaCaliente ? potencia * 1.20 : potencia;
        return pot * horas;
    }
}
```

Lo que hay que saber de herencia

- `extends` → una sola clase padre (Java no permite herencia múltiple entre clases).
- `super(params)` → llama al constructor del padre. Debe ser la **primera línea** del constructor hijo.
- `super.metodo()` → llama a la versión del método del padre.
- `@Override` → marca que estás sobrescribiendo. El compilador verifica la firma.
- Los atributos `private` del padre **no se heredan directamente**; se accede a ellos con los getters/setters del padre.
- Los atributos `protected` sí son accesibles directamente desde la subclase.

Tipos de herencia (pregunta típica de teoría)

Tipo	Descripción	Esquema
Única	Una subclase hereda de una superclase	`B extends A`
Multinivel	Cadena: C hereda de B, B hereda de A	`A → B → C`
Jerárquica	Varias subclases del mismo padre	`B, C, D extends A`
Múltiple	Una clase hereda de varias. **Solo posible con interfaces en Java**	`C implements A, B`

CLASES ABSTRACTAS (`abstract class`)

Una clase abstracta **no se puede instanciar**. Define estructura común y obliga a las subclases a implementar ciertos métodos.

```
public abstract class FiguraGeometrica {
    protected String nombre;
    protected String color;
    public FiguraGeometrica(String nombre, String color) {
        this.nombre = nombre;
        this.color = color;
    }
    // Método concreto: ya tiene implementación, las subclases lo heredan tal cual
    public void mostrarInformacion() {
        System.out.println("Figura: " + nombre + " | Color: " + color);
    }
    // Métodos abstractos: sin cuerpo, OBLIGAN a las subclases a implementarlos
    public abstract double calcularArea();
    public abstract double calcularPerimetro();
    public abstract void dibujar();
}
```

```
public class Circulo extends FiguraGeometrica {
    private double radio;
    public Circulo(String nombre, String color, double radio) {
        super(nombre, color);
        this.radio = radio;
    }
    @Override
    public double calcularArea() { return Math.PI * radio * radio; }
    @Override
    public double calcularPerimetro() { return 2 * Math.PI * radio; }
    @Override
    public void dibujar() {
        System.out.println("Círculo de radio " + radio);
    }
}
```

Lo que hay que saber de clases abstractas

- `abstract class` → no se puede hacer `new FiguraGeometrica(...)`.
- Método abstracto → termina en `;`, sin llaves `{}`.
- Una subclase concreta **debe implementar todos los métodos abstractos**. Si no los implementa, debe ser abstracta también.
- Puede tener constructor (lo llaman las subclases con `super`).
- Puede tener atributos normales, `private`, `protected`, `static`...
- Puede mezclar métodos abstractos y concretos.
- **Cuándo usarla:** cuando varias clases comparten código real (atributos, métodos concretos) pero también necesitan comportamiento específico obligatorio.

INTERFACES (`interface` / `implements`)

Una interfaz es un **contrato**: define qué métodos debe tener la clase, sin implementarlos (salvo `default`).

```
public interface Empleado {
    double calcularSalarioMensual();   // implícitamente public abstract
    void mostrarDetalles();
}
```

```
public class EmpleadoRegular implements Empleado {
    private String nombre;
    private double salarioBase;
    public EmpleadoRegular(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }
    @Override
    public double calcularSalarioMensual() { return salarioBase; }
    @Override
    public void mostrarDetalles() {
        System.out.println(nombre + " | Salario: " + calcularSalarioMensual());
    }
}
```

Clase abstracta que implementa una interfaz

La clase abstracta puede implementar **algunos** métodos de la interfaz y dejar otros para las subclases:

```
public abstract class EmpleadoBase implements Empleado {
    protected String nombre;
    protected int id;
    protected double salarioBase;
    public EmpleadoBase(String nombre, int id, double salarioBase) {
        this.nombre = nombre;
        this.id = id;
        this.salarioBase = salarioBase;
    }
    @Override
    public void mostrarDetalles() {             // implementa este de la interfaz
        System.out.println("Nombre: " + nombre + " | ID: " + id);
    }
    // calcularSalarioMensual() NO se implementa aquí → las subclases lo hacen
}

public class Gerente extends EmpleadoBase {
```

```
    private double bonoAnual;
    public Gerente(String nombre, int id, double salarioBase, double bonoAnual) {
        super(nombre, id, salarioBase);
        this.bonoAnual = bonoAnual;
    }
    @Override
    public double calcularSalarioMensual() { return salarioBase + bonoAnual; }
}
```

Lo que hay que saber de interfaces

- `interface` → se implementa con `implements`, no con `extends`.
- Una clase puede implementar **varias interfaces**: `class X implements A, B, C`.
- Una interfaz puede extender otras interfaces: `interface C extends A, B`.
- Todos los métodos son `public abstract` por defecto (no hace falta escribirlo).
- Todos los atributos son `public static final` (constantes).
- Las interfaces **no tienen constructor** ni estado de instancia.
- Método `default`: tiene implementación, las clases pueden sobrescribirlo o no.
- **Cuándo usarla:** para comportamientos "transversales" independientes de la jerarquía (`Encendible`, `Imprimible`, `Comparable`…).

POLIMORFISMO

Usar una variable del tipo padre/interfaz que apunta a objetos de distintas subclases. Al llamar a un método, se ejecuta el de la clase **real** del objeto.

```
FiguraGeometrica[] figuras = {
    new Circulo("c1", "rojo", 5),
    new Rectangulo("r1", "azul", 4, 3)
};
for (FiguraGeometrica f : figuras) {
    System.out.println(f.calcularArea());  // cada una ejecuta su versión
}
// Con interfaz:
Empleado e = new Gerente("Ana", 1, 2000, 500);
e.calcularSalarioMensual();  // ejecuta Gerente.calcularSalarioMensual()
```

COMPARATIVA RÁPIDA

	Clase Abstracta	**Interfaz**
Palabra clave	`abstract class`	`interface`
Se usa con	`extends`	`implements`
¿Instanciable?	No	No
Constructor	Sí	No
Atributos normales	Sí	No (solo constantes)
Métodos concretos	Sí	Solo `default`
Herencia múltiple	No (una sola)	Sí (varias)
¿Cuándo?	Código compartido + obligar implementación	Contrato de comportamiento

---

ERRORES TÍPICOS DE EXAMEN

1. **Instanciar clase abstracta**: `new FiguraGeometrica()` → error de compilación.
2. **No implementar todos los métodos de la interfaz** en una clase concreta → error de compilación.
3. **Olvidar `super(...)` en el constructor hijo** → el compilador lo exige si el padre no tiene constructor sin parámetros.
4. **Confundir `extends` con `implements`**: clases usan `extends`, interfaces usan `implements`.
5. **`@Override` con firma incorrecta**: si no coincide exactamente el nombre y parámetros, se crea un método nuevo en vez de sobrescribir.
6. **Acceder a atributo `private` del padre directamente**: no se puede, hay que usar el getter.

ESTRUCTURA TÍPICA DE EJERCICIO

```
Interfaz / Clase abstracta
    └── Clase abstracta intermedia (implements interfaz, implementa parte)
            ├── Subclase concreta A (implementa lo que falta)
            └── Subclase concreta B (implementa lo que falta, con variante)
```

Ejemplo real de prácticas:

```
Empleado (interface)
    └── EmpleadoBase (abstract, implements Empleado)
            ├── EmpleadoRegular (salario = salarioBase)
            └── Gerente (salario = salarioBase + bonoAnual)

FiguraGeometrica (abstract class)
    ├── Circulo
    └── Rectangulo
        └── Cuadrado (extends Rectangulo)
```